背景

当前的 AI 编码助手本质上是**无状态的**。每个新会话从零开始，它不记得你上次的架构决策，不记得你定下的接口协议，不记得你说过"这个模块暂时不动"。

这导致三个典型问题：

上下文漂移：同一个需求，在不同会话里可能得到完全不同的实现方案，因为每次的"背景知识"不一样。

重复对齐成本：每次新会话都要花时间把历史决策重新交代给 AI，这个隐性成本随项目复杂度指数上升。对于复杂的系统这个成本尤其高。

意图丢失：AI 生成的代码往往技术上可行，但脱离了你系统设计的大局。你要的是"符合现有记忆分层架构的检索接口"，AI 给你的是"一个能用的检索函数"。

传统解法是靠 CLAUDE.md 或系统提示词，把规范塞进上下文。这一定程度上有效，但随着项目增长，维护一个冗长的提示词文件既低效又脆弱。

OpenSpec 的解题思路

对话是短暂的，但系统设计是持续的。 OpenSpec 从这个认知出发：与其每次靠对话重建上下文，不如把项目说明文档持久化在代码库里。

OpenSpec 是一个**规格驱动开发（Spec-Driven Development）框架**，核心思想是：**用结构化文档作为 AI 的持久化上下文**。AI 每次工作时读取这份说明，保证输出始终对齐你的设计意图，而不是靠对话历史猜测。

两层存储结构

OpenSpec 在项目根目录创建 openspec/ 文件夹，里面分两个区：

```
openspec/
├── specs/              # 系统现状：你的系统"现在是什么样"
│   ├── auth/
│   │   ├── login.md       # 认证流程规格
│   │   └── permissions.md  # 权限模型规格
│   ├── api/
│   │   └── rate-limit.md   # 限流策略规格
│   └── payments/
└── changes/            # 变更提案：你"打算改什么"
    ├── add-oauth-login/
    ├── refactor-rate-limit/
    └── archive/
```

specs/ 是系统的"真相来源"（source of truth），用 RFC 2119 关键字（MUST、SHOULD、MAY）描述系统行为，配上 Given/When/Then 格式的可测试场景。

changes/ 是变更的沙盒。每个功能或修改在这里有独立的文件夹，开发完成后归档，并把变更内容合并回 specs。

Delta Specs：只描述变化

不同于"重写整份文档"，OpenSpec 用 **Delta Specs** 机制标记变更：

```
## 用户认证

<!-- ADDED -->
- 系统 MUST 支持 OAuth 2.0 授权码流程
- 登录态 SHOULD 在 7 天后自动过期

<!-- MODIFIED -->
- ~~Token 存储在 localStorage~~
- Token 存储在 HttpOnly Cookie，防止 XSS 读取
```

变更完成归档时，ADDED 的内容追加到主 specs，MODIFIED 的内容替换旧规则，REMOVED 的内容删除。历史上下文完整保留在 archive 里。

四个 Artifacts 驱动开发

每个 change 文件夹包含四个文档，构成从"想做"到"做完"的完整链路：

Artifact	解决什么问题
proposal.md	为什么要做这个变更，范围是什么
specs/	变更后系统行为应该是什么（Delta Specs）
design.md	技术方案怎么实现
tasks.md	分解为可执行的 checkbox 任务列表

AI 在执行 /opsx:apply 时，会读取这四份文档，理解完整意图后再写代码。

实际上手

安装

需要 Node.js 20.19+。

```
npm install -g @fission-ai/openspec@latest
openspec --version  # 验证安装
```

初始化项目

```
cd your-game-project
openspec init
```

初始化会创建 openspec/ 目录结构和 config.yaml，同时在 Claude Code 里注册 /opsx:* 系列命令。

命令的工作原理

/opsx:* 命令本质上是 **skill 文件**，不是 CLI 内置逻辑。openspec init 会把对应的 skill 文件写入 .claude/skills/openspec-*/SKILL.md，命令文件写入 .claude/commands/opsx/，AI 工具直接读取这些文件来执行工作。

这个设计有两个关键含义：

跨工具兼容：24 个 AI 编码工具（Claude Code、code、Cursor、Windsurf、GitHub Copilot 等）都通过同一份文件驱动，区别只是安装路径不同。

动态指令生成：每次执行命令时，CLI 实时查询 artifact 状态（openspec status --json），根据依赖图（DAG）决定当前可以做什么、什么被阻塞。你不需要手动追踪"现在该到哪一步了"。

Artifact 的依赖关系如下：

```
proposal → specs ────┐
                     ├→ tasks → apply → archive
proposal → design ───┘
```

/opsx:continue 就是沿着这条链路逐步推进，每次生成一个节点。

默认命令 vs 扩展命令

OpenSpec 默认只开放**核心 profile**（3 个命令）：

命令	作用
/opsx:propose	一键生成所有规划 artifacts
/opsx:apply	按任务列表实现代码
/opsx:archive	归档，合并 delta specs

扩展命令默认隐藏，需要手动开启：

```
openspec config profile    # 切换到 expanded profile
openspec update            # 重新安装 skill 和 command 文件
```

开启后新增的命令：

命令	作用	典型场景
/opsx:explore	调研，不创建 change	需求不清晰，先摸底
/opsx:new	创建空 change 脚手架	精细控制每个 artifact
/opsx:continue	生成依赖链上的下一个 artifact	逐步推进，每步可审查
/opsx:ff	批量生成所有规划 artifacts	需求明确时的快速路径
/opsx:verify	验证实现是否符合规格	归档前的质量门控
/opsx:sync	将 delta specs 合并进主 specs	长期迭代的 change
/opsx:bulk-archive	批量归档多个 change	并行功能开发收尾
/opsx:onboard	交互式教程	初次上手

什么时候需要扩展命令：核心 profile 适合大多数场景。当你需要逐步审查每个 artifact、处理并行变更冲突、或者在归档前做系统性验证时，再开启扩展 profile。

工作流一：快速提案（80% 场景）

需求清晰时，一条命令完成所有规划文档：

```
> /opsx:propose 帮我实现批量删除接口

● 好的，我来帮你创建"批量删除接口"的完整提案。先创建变更目录。

● Bash(cd /d/D/code/mem_minus && openspec new change "batch-delete-api")
  └ Running…
```

Claude Code 会自动生成 openspec/changes/batch-delete-api/ 目录，并填充四个 artifacts。

创建的 Artifacts

Artifact	说明
proposal.md	动机：缺少按 ID 列表批量删除能力，客户端需循环调用效率低
design.md	技术方案：POST 方法、向量存储层 batch_delete、上限 100 条、非事务性部分成功
specs/batch-delete/spec.md	4 个需求规格：批量删除端点、实验组支持、向量存储批量能力、请求追踪
tasks.md	8 个实现任务，分 4 组：向量存储层 → Memory 编排层 → API 端点 → 测试

生成的 proposal.md 大致如下：

```
## Why

当前删除接口只支持单条删除（`DELETE /memories/{memory_id}`）或基于过滤条件的批量删除（`DELETE /memories`），缺少按指定 ID 列表批量删除的能力。业务场景中常需要一次性删除多条指定记忆（如用户在 UI 勾选多条后批量删除），现有方案需要客户端循环调用单条删除接口，效率低且网络开销大。

## What Changes

- 新增 `POST /batch_delete` 接口，接受 memory_id 列表，一次性删除多条指定记忆
- 在 `Memory` 类中新增 `batch_delete(memory_ids)` 方法，支持按 ID 列表批量删除
- 在 `VectorStoreBase` 及各实现（Milvus、Qdrant、FAISS）中新增 `batch_delete(vector_ids)` 方法，利用底层 SDK 的批量删除能力提升性能
- 返回删除结果摘要（成功数、失败数、失败 ID 列表）

## Capabilities

### New Capabilities
- `batch-delete`: 按指定 memory_id 列表批量删除记忆的能力，包含 API 端点、Memory 编排层和向量存储层的批量删除实现

### Modified Capabilities
```

Impact

- **API**: 新增 `POST /batch_delete` 端点，不影响现有删除接口
- **代码**: 涉及 `server/server.py`、`mem/memory/memory.py`、`mem/vector_stores/base.py` 及各向量存储实现
- **依赖**: 无新依赖，复用现有 SDK 的批量操作能力
- **兼容性**: 纯新增接口，无 breaking change

审查 artifacts，人工确认没问题后再执行：

/opsx:apply

AI 按照 tasks.md 的 checkbox 逐项实现，完成后标记为已完成。

```
> /opsx:apply

● Implementing: batch-delete-api (schema: spec-driven)

● Bash(cd /d/D/code/mem_minus && openspec status --change "batch-delete-api" --json)
  └ - Loading change status...
    {
      "changeName": "batch-delete-api",
    … +28 lines (ctrl+o to expand)

● Bash(cd /d/D/code/mem_minus && openspec instructions apply --change "batch-delete-api" --json)
  └ - Generating apply instructions...
    {
      "changeName": "batch-delete-api",
    … +37 lines (ctrl+o to expand)

● Progress: 0/9 tasks complete
```

验证实现与规格一致：

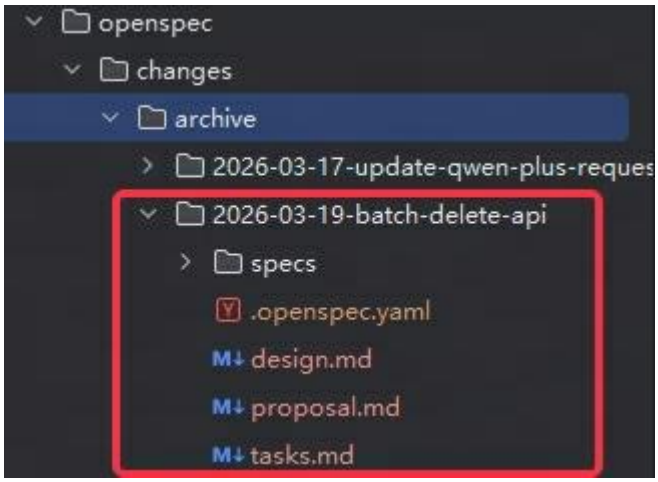
/opsx:verify

```
Completed This Session

1. 向量存储层
- 1.1 mem/vector_stores/base.py - VectorStoreBase.batch_delete() 默认实现（循环调用 delete()）
- 1.2 mem/vector_stores/milvus.py - Milvus 原生批量删除 client.delete(ids=list)
- 1.3 mem/vector_stores/qdrant.py - Qdrant PointIdsList 批量删除
- 1.4 mem/vector_stores/faiss.py - 批量移除后单次写盘

2. Memory 编排层
- 2.1 mem/memory/memory.py - Memory.batch_delete() 方法，带 fallback 逐条删除和结构化结果返回

3. API 端点
- 3.1 BatchDeleteRequest Pydantic 模型
- 3.2 POST /batch_delete 端点（空列表 400、超 100 条 400、experiment_id 支持）
- 3.3 sid 追踪日志

4. 测试
- 4.1 tests/unit_test/test_batch_delete.py - 5 个测试用例

All tasks complete! You can archive this change with /opsx:archive.
```

归档：

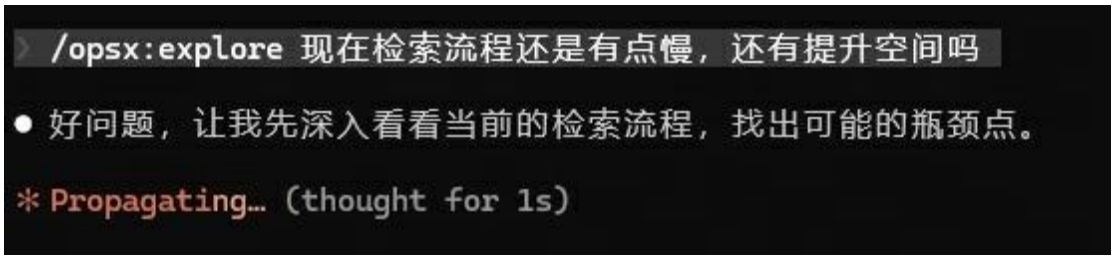
/opsx:archive

Delta Specs 合并进主 specs，change 文件夹带日期前缀移入 archive。



工作流二：探索式开发（需求模糊时）

不确定要怎么设计时，先探索再提案：



这个命令不创建 change，只做分析：调查现有代码库、对比方案、生成可视化图表。弄清楚方向后，再 /opsx:new 创建 change，用 /opsx:continue 逐个生成 artifacts，每次生成后可以审查修改，再继续下一个。
或者直接复用工作流一。

工作流三：并行多个变更

同时进行多个功能开发时：

```
openspec list          # 查看所有活跃 changes
openspec show add-episodic-memory  # 查看具体 change 详情
```

切换工作焦点时：

```
/opsx:apply add-npc-personality   # 指定 change 名称
```

多个 change 完成后批量归档：

```
/opsx:bulk-archive
```

OpenSpec 会检测冲突并按时间顺序处理。

什么时候更新 Artifacts，什么时候重新来过

开发过程中需求会变，OpenSpec 推荐的判断标准：
更新现有 artifacts：意图相同，只是执行细节变了（比如发现某个接口要调整参数）。直接修改 design.md 或 tasks.md，然后继续 apply。
新建 change：意图根本性变化（比如"情节记忆"演变成"全面记忆重构"）。原有 change

归档，新开一个。
这个原则很重要：**不要让一个 change 的范围无限膨胀**。保持每个 change 单一职责，归档后才能形成清晰的系统演进历史。

与传统方式的差别

	对话驱动开发	OpenSpec
上下文如何传递	靠对话历史	靠版本化文档
跨会话一致性	取决于你的记忆	由 specs 保证
需求变更	重新描述，AI 可能漂移	更新 artifacts，AI 重新对齐
团队协作	每人自己的 prompts	共享 specs，可 code review
历史审计	无	archive 记录完整演进

对于长期项目，specs 层的价值会随时间积累——三个月后的新同事，或者一年后的你，都能通过 specs 快速理解系统当前状态，而不用翻几百条对话记录。

开始使用

如果你的项目已经在用 Claude Code，接入 OpenSpec 的成本极低：

npm install -g @fission-ai/openspec@latest
在项目根目录 openspec init
下一个新功能，用 /opsx:propose 替代直接让 AI 写代码

不需要一次性把所有存量设计迁移进 specs。从下一个 change 开始，让 specs 随项目自然生长。
如果你想先感受一下完整流程，可以在项目里运行：

```
/opsx:onboard
```

这个命令会用你真实的代码库，引导你走一遍完整的提案→实现→归档流程。